

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
КАФЕДРА АВТОМАТИЗАЦІЇ ТА ІНФОРМАЦІЙНИХ СИСТЕМ

ЗВІТ

ЗВІТ З ЛАБОРАТОРНИХ РОБІТ
З НАВЧАЛЬНОЇ ДИСЦИПЛІНИ
«КРОС-ПЛАТФОРМНЕ ПРОГРАМУВАННЯ»

Виконав студент групи КН-23-1

Полинько Ігор Миколайович

Перевірів: старший викладач кафедри АІС Бельська В. Ю.

Кременчук 2026

ЛАБОРАТОРНА РОБОТА № 7

Тема: Maven. Тестування програмного коду: JUnit5. Проект Lombok.

Мета: отримати навички використання збирача Maven для підвантаження фреймворків, що дозволять скоротити написання програмного коду та тестувати основні модулі створених проектів шляхом покрити їх тестами фреймворку JUnit.

Порядок виконання роботи:

Завдання 1:

1. Створіть порожній Maven проект

1.1 groupId - brainacad-org

1.2 artifactId - test-lib

1.3 version - 1.0

1.4 створіть блок properties в шапці конфігурації. Додайте налаштування для компілятора (версія Java)

```
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
</properties>
```

2. Додайте залежності у проект

2.1 Додайте блок dependencies

2.2 Додайте залежність на бібліотеку

2.2.1 Відкриваємо сайт <https://mvnrepository.com/>

2.2.2 Шукаємо бібліотеку, наприклад guava

2.2.3 Обираємо необхідну залежність (Guava: Google Core Libraries For Java)

2.2.4 Копіюємо блок з залежністю <dependency>...</dependency> у свою конфігурацію

2.2.5 Імпортуємо залежність у IDEA (якщо не включено auto-import)

```
<dependencies>
  <dependency>
    <groupId>com.google.guava</groupId>
    <artifactId>guava</artifactId>
    <version>33.6.0-jre</version>
  </dependency>
</dependencies>
```

3. Внесіть версію Guava у окремі налаштування

3.1 у блок properties в шапку конфігурації додайте змінну guava.version зі значенням потрібної версії (28.0-jre)

3.2 Замініть версію у бібліотеці на цей параметр

```
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <guava>33.6.0-jre</guava>
</properties>

<dependencies>
  <dependency>
    <groupId>com.google.guava</groupId>
    <artifactId>guava</artifactId>
    <version>${guava}</version>
  </dependency>
</dependencies>

</project>
```

4. Перевірте роботу нової бібліотеки.

4.1 Створіть клас TestMaven (з main методом)

4.2 Перевірте роботу спеціального утилітного класу Lists

```
List<String> fruits = Lists.newArrayList("orange", "banana", "kiwi");
fruits.forEach(System.out::println);

List<String> reverseFruits = Lists.reverse(fruits);
reverseFruits.forEach(System.out::println);
```

4.3 Перевірте роботу спеціальної Map - Multimap

```
Multimap<String, String> map = ArrayListMultimap.create();
map.put("key", "firstValue");
map.put("key", "secondValue");
System.out.println(map);
```

4.4 Перевірте import-и. (Цих імпортів немає в Java Core)

```
import com.google.common.collect.ArrayListMultimap;
import com.google.common.collect.Lists;
import com.google.common.collect.Multimap;
```

4.5 Запустіть та перевірте роботу

```

orange
banana
kiwi
kiwi
banana
orange
{key=[firstValue, secondValue]}

Process finished with exit code 0

```

Рисунок 7.1 – Результат роботи скрипту TestMaven

5. Додайте 2 профілі збірки

5.1 Додайте тег `<profiles>...</profiles>`

5.2 Додайте тег `<profile>` с id = «dev»

5.3 Додайте тег `<profile>` с id = «prod»

5.4 Для профілю dev додайте налаштування профілю за замовчанням

5.5 Відкрийте вкладку maven в IDEA та перевірте наявність профілів

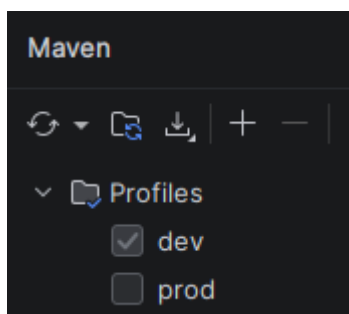


Рисунок 7.2 – Наявність профілів у вкладці Maven

5.6 Додайте для кожного профілю блок

5.7 Додайте опції:

Для dev профілю додайте повідомлення:

```
<test.message>Maven hello from dev</test.message>
```

Для dev профілю додайте повідомлення:

```
<test.message>Maven hello from prod</test.message>
```

5.8 Створіть файл з налаштуваннями проекту

5.8.1 У папці resources створіть файл config.properties

5.8.2 У файлі оголосіть 2 опції (одна самостійна, друга - з прив'язкою до mvn конфігурації)

5.9 Перевірити роботу опції

5.9.1 Реалізувати завантаження налаштувань на момент старту системи

```
public static void main(String[] args) {
    Properties prop = new Properties();

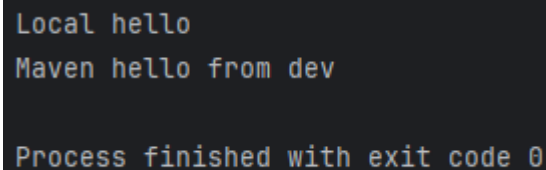
    try (InputStream resourceAsStream = TestMaven.class.getClassLoader()
        .getResourceAsStream("config.properties")) {

        prop.load(Objects.requireNonNull(resourceAsStream));

        System.out.println(prop.getProperty("props.local.hello"));
        System.out.println(prop.getProperty("props.mvn.hello"));

    } catch (IOException e) {
        System.err.println("Unable to load properties file:
config.properties");
    }
}
```

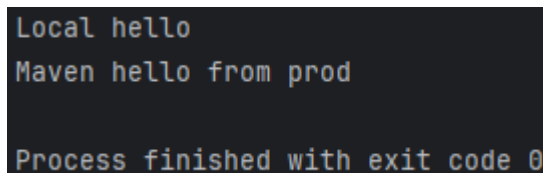
5.9.2 Вивести на екран значення опцій.



```
Local hello
Maven hello from dev

Process finished with exit code 0
```

Рисунок 7.3 – Результат виводу опцій на профілі dev



```
Local hello
Maven hello from prod

Process finished with exit code 0
```

Рисунок 7.4 – Результат виводу опцій на профілі prod

5.9.3 Запустити програму та перевірити значення опцій

5.9.4 Змінити профіль, запустити програму та перевірити значення опцій

5.9.5 Додати плагін, для того, щоб побудувати зв'язок файлу налаштувань з помніком.

```
<build>
  <plugins>
    <plugin>
```

```

<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-resources-plugin</artifactId>
<version>3.5.0</version>
<configuration>
  <encoding>UTF-8</encoding>
</configuration>
</plugin>
</plugins>

<resources>
  <resource>
    <directory>src/main/resources</directory>
    <filtering>true</filtering>
  </resource>
</resources>
</build>

```

6. Виконати команду clean та install

6.1 Перевірити наявність зібраного артефакту (jar)

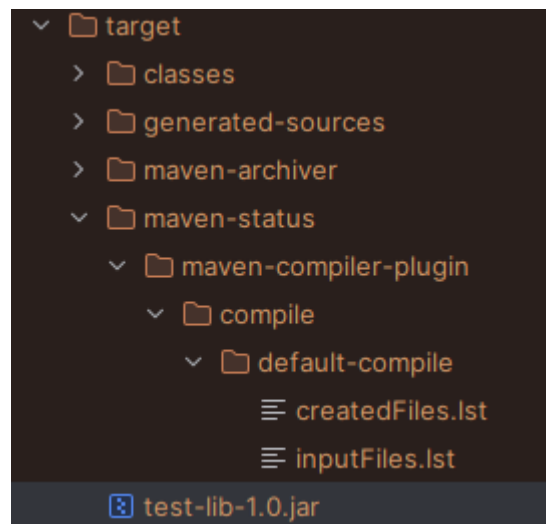


Рисунок 7.5 – Наявність зібраного артефакту

6.2 Перевірити наявність бібліотек у локальному maven-repository

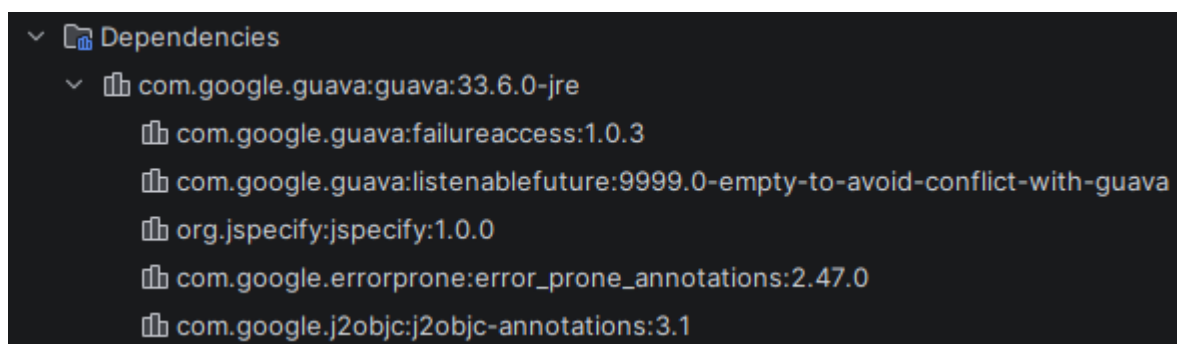


Рисунок 7.6 – Наявність бібліотек у локальному maven-repository

7. Додати простий плагін до проекту

7.1 Додати плагін у блок build

```
<build>
<plugins>
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <configuration>
      <source>8</source>
      <target>8</target>
    </configuration>
  </plugin>
</plugins>
</build>
```

7.2 Подивитися список плагінів



Рисунок 7.7 – Наявність бібліотек у локальному maven-repository

Завдання 2.

Використовуючи фреймворки Lombok та JUnit5 виконайте наступне завдання:

Розробити систему "Автобаза". Щодня з'являються випадково заявки на перевезення, які Диспетчер розподіляє на Рейси між Водіями та призначає для цього Автомобіль.

Заявка на рейс повинна містити пункт призначення, кількість та тип вантажу.

Водій на рейс визначається виходячи із стажу, необхідного для перевезення зазначеного у заявці товару, складності керування Автомобілем та Довжини шляху.

Автомобіль на рейс визначається з принципу оптимізації ваги, що перевозиться, і вантажопідйомності автомобіля.

Під час подорожі автомобіль може зламатися. Водій може зробити заявку на ремонт. Після досягнення кінцевого пункту водій отримує певну суму виплат. При

цьому звільняється автомобіль та водій, і вони доступні для виконання наступних заявок.

Водій робить відмітку про виконання Рейсу та стан Автомобіля.

Під час розробки програми використовувати контейнерні класи стандартної бібліотеки. Зробити логування усіх подорожей у файл через файлові потоки. Зробити збір статистики скільки вантажів було перевезено кожним водієм, скільки вантажів перевезено до конкретного пункту призначення, водії з найбільшим заробітком тощо.

Протестуйте класи та методи, що реалізують логіку роботи зі створеними класами моделі даних за допомогою тестів фреймворку JUnit5.

Завдання 2:

pom.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>brainacad-org</groupId>
  <artifactId>test-lib</artifactId>
  <version>1.0</version>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-resources-plugin</artifactId>
        <version>3.2.0</version>
        <configuration>
          <resources>
            <resource>
              <directory>src/main/resources</directory>
              <filtering>true</filtering>
            </resource>
          </resources>
        </configuration>
      </plugin>

      <plugin> <!-- підключення плагінів-->
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.15.0</version>
        <configuration> <!-- налаштування плагіна-->
          <source>21</source>
          <target>21</target>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```



```

        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.1.2</version>
        </plugin>
    </plugins>
</build>

<profiles>
    <profile>
        <id>dev</id>
        <activation>
            <activeByDefault>true</activeByDefault>
        </activation>
        <properties>
            <test.message>Maven hello from dev</test.message>
        </properties>
    </profile>
    <profile>
        <id>prod</id>
        <properties>
            <test.message>Maven hello from prod</test.message>
        </properties>
    </profile>
</profiles>

<properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <guava>33.5.0-jre</guava>
</properties>

<dependencies>
    <dependency>
        <groupId>com.google.guava</groupId>
        <artifactId>guava</artifactId>
        <version>${guava}</version>
    </dependency>

    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>6.0.3</version>
    </dependency>

    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <version>1.18.42</version>
        <scope>provided</scope>
    </dependency>

    <dependency>
        <groupId>com.github.javafaker</groupId>
        <artifactId>javafaker</artifactId>
        <version>1.0.2</version>
    </dependency>
</dependencies>
</project>

```

Driver.java:

```
package driver;

import lombok.*;

@Data
public class Driver {
    @NonNull
    private String name;
    @NonNull
    private String surname;
    @NonNull
    private String phone;
    @NonNull
    private Integer experience;

    // Заробіток водія
    private double earnings = 0.0;

    // Кількість виконаних рейсів
    private int completedTrips = 0;

    // Додати гроші до заробітку
    public void addEarnings(double amount) {
        earnings += amount;
    }

    // Збільшити кількість виконаних рейсів
    public void incrementCompletedTrips() {
        completedTrips++;
    }

    @Override
    public String toString() {
        return ""
            + Ім'я: %s
            + Прізвище: %s
            + Телефон: %s
            + Стаж: %d
            + Заробіток: %.2f грн
            + Виконано рейсів: %d
            + ""
            .formatted(name, surname, phone, experience, earnings,
            completedTrips).trim();
    }
}
```

Address.java:

```
package trip;

public record Address(String address) {
    @Override
    public String toString() {
        return "Пункт призначення: " + address;
    }
}
```

CargoType.java:

```
package trip;

import lombok.Getter;
import lombok.RequiredArgsConstructor;

@Getter
@RequiredArgsConstructor
public enum CargoType {
    GENERAL("звичайний", 1, 1.0),
    FRAGILE("крихкий", 2, 1.2),
    HEAVY("важкий", 3, 1.5),
    DANGEROUS("небезпечний", 4, 2.0);

    private final String ukrainianName;
    private final int requiredExperience;
    private final double paymentMultiplier;

    @Override
    public String toString() {
        return ukrainianName;
    }
}
```

CargoType.java:

```
package trip;

import lombok.Data;
import lombok.NonNull;
import driver.Driver;
import vehicle.Vehicle;

import java.time.LocalDate;

@Data
public class Trip {
    // Призначений автомобіль для рейсу
    private Vehicle vehicle;

    // Призначений водій для рейсу
    private Driver driver;

    // Дата початку рейсу
    private LocalDate startDate;

    // Дата завершення рейсу
    private LocalDate endDate;

    // Відстань маршруту
    @NonNull
    private Double distance;

    // Пункт призначення
    @NonNull
    private Address address;

    // Вага вантажу
    @NonNull
    private Integer neededWeight;

    // Тип вантажу
    @NonNull
    private CargoType cargoType;
}
```

```

// Оплата за рейс
private Double payment;

// Розрахунок необхідного стажу водія
public Integer getNeededExperience() {
    int byCargo = cargoType.getRequiredExperience();
    int byDistance = Math.max(1, (int) Math.ceil(distance / 250.0));
    int byWeight = Math.max(1, neededWeight / 250);
    return byCargo + byDistance + byWeight;
}

// Розрахунок вартості рейсу
public double calculatePayment() {
    double base = neededWeight * distance * 0.5;
    return base * cargoType.getPaymentMultiplier();
}

@Override
public String toString() {
    return ""
        + Рейс:
        + Автомобіль:
        + %s
        + Водій:
        + %s
        + Дата початку: %s
        + Дата завершення: %s
        + Відстань: %.2f км
        + %s
        + Вага вантажу: %d кг
        + Тип вантажу: %s
        + Оплата: %.2f грн
        + ""formatted(
            vehicle == null ? "    не призначено" :
indent(vehicle.toString(), 4),
            driver == null ? "    не призначено" :
indent(driver.toString(), 4),
            startDate,
            endDate,
            distance,
            address,
            neededWeight,
            cargoType,
            payment == null ? 0.0 : payment
        ).trim();
}

// Допоміжний метод для красивого відступу в багаторядковому тексті
private String indent(String text, int spaces) {
    String prefix = " ".repeat(spaces);
    return text.replaceAll("(?m)^", prefix);
}
}

```

DefaultVehicle.java:

```

package vehicle;

import lombok.*;

@ToString
@EqualsAndHashCode
@AllArgsConstructor

```

```

@NoArgsConstructor
public class DefaultVehicle implements Vehicle {
    private String name = "";
    private Integer maxWeight = 0;
    private Integer drivingComplexity = 1;
    private VehicleStatus status = VehicleStatus.OK;

    @Override
    public String getName() {
        return name;
    }

    @Override
    public Integer getMaxWeight() {
        return maxWeight;
    }

    @Override
    public Integer getDrivingComplexity() {
        return drivingComplexity;
    }

    @Override
    public VehicleStatus getStatus() {
        return status;
    }

    @Override
    public void setStatus(VehicleStatus status) {
        this.status = status;
    }

    @Override
    public String toString() {
        return ""
            + Назва: %s
            + Вантажопідйомність: %d
            + Складність керування: %d
            + Статус: %s
            + ""
            .formatted(name, maxWeight, drivingComplexity,
status.getUkrainianName()).trim();
    }
}

```

Vehicle.java:

```

package vehicle;

public interface Vehicle {
    String getName();
    Integer getMaxWeight();
    Integer getDrivingComplexity();
    VehicleStatus getStatus();
    void setStatus(VehicleStatus status);
}

```

VehicleStatus.java:

```

package vehicle;

public enum VehicleStatus {
    OK("справний"),
    REPAIR("на ремонті"),
}

```

```

        BROKEN("зламаний");

        private final String ukrainianName;

        VehicleStatus(String ukrainianName) {
            this.ukrainianName = ukrainianName;
        }

        public String getUkrainianName() {
            return ukrainianName;
        }
    }
}

```

DispatcherService.java:

```

import lombok.*;
import driver.Driver;
import trip.Trip;
import vehicle.Vehicle;
import vehicle.VehicleStatus;

import java.nio.file.Path;
import java.time.LocalDate;
import java.util.*;
import java.util.stream.Collectors;

@Getter
@ToString
@EqualsAndHashCode
@NoArgsConstructor
public class DispatcherService {
    // Черга заявок, які ще не оброблені
    private Queue<Trip> pendingTrips = new ArrayDeque<>();

    // Список доступних автомобілів
    private List<Vehicle> vehicles = new ArrayList<>();

    // Список доступних водіїв
    private List<Driver> drivers = new ArrayList<>();

    // Рейси, які зараз у процесі виконання
    private Set<Trip> tripsInProgress = new HashSet<>();

    // Завершені рейси
    private List<Trip> completedTrips = new ArrayList<>();

    // Логування завершених рейсів у файл
    private StreamLogger<Trip> tripLogger = new
StreamLogger<>(Path.of("trips.log"));

    @Builder
    public DispatcherService(Queue<Trip> pendingTrips,
                             List<Vehicle> vehicles,
                             List<Driver> drivers,
                             Set<Trip> tripsInProgress,
                             List<Trip> completedTrips,
                             StreamLogger<Trip> tripLogger) {
        this.pendingTrips = pendingTrips != null ? pendingTrips : new
ArrayDeque<>();
        this.vehicles = vehicles != null ? vehicles : new ArrayList<>();
        this.drivers = drivers != null ? drivers : new ArrayList<>();
        this.tripsInProgress = tripsInProgress != null ? tripsInProgress :
new HashSet<>();
        this.completedTrips = completedTrips != null ? completedTrips : new

```

```
ArrayList<>();
    this.tripLogger = tripLogger != null ? tripLogger : new
StreamLogger<>(Path.of("trips.log"));
}

// Запис рейсу в лог-файл
public void logTrip(Trip trip) {
    tripLogger.append(trip);
}

// Видалити авто зі списку доступних
public void removeVehicle(Vehicle vehicle) {
    vehicles.remove(vehicle);
}

// Видалити водія зі списку доступних
public void removeDriver(Driver driver) {
    drivers.remove(driver);
}

// Додати авто
public void addVehicle(Vehicle vehicle) {
    vehicles.add(vehicle);
}

// Додати водія
public void addDriver(Driver driver) {
    drivers.add(driver);
}

// Додати нову заявку
public void addTrip(Trip trip) {
    pendingTrips.add(trip);
}

// Водій просить ремонт
public void driverRequestRepair(Driver driver) {
    tripsInProgress.stream()
        .filter(trip -> trip.getDriver() != null &&
trip.getDriver().equals(driver))
        .forEach(trip ->
trip.getVehicle().setStatus(VehicleStatus.REPAIR));
}

// Водій підтверджує, що авто в нормі
public void driverRequestOk(Driver driver) {
    tripsInProgress.stream()
        .filter(trip -> trip.getDriver() != null &&
trip.getDriver().equals(driver))
        .forEach(trip ->
trip.getVehicle().setStatus(VehicleStatus.OK));
}

// Водій повідомляє, що авто зламалось
public void driverReportBrokenVehicle(Driver driver) {
    tripsInProgress.stream()
        .filter(trip -> trip.getDriver() != null &&
trip.getDriver().equals(driver))
        .forEach(trip ->
trip.getVehicle().setStatus(VehicleStatus.BROKEN));
}

// Призначення наступного рейсу
public boolean assignNextTrip() {
    if (pendingTrips.isEmpty() || vehicles.isEmpty() ||
```

```

drivers.isEmpty()) {
    return false;
}

// Дивимось наступну заявку, але не забираємо її, поки не знайдемо
ресурс
Trip trip = pendingTrips.peek();

Optional<Vehicle> vehicleOptional = findOptimalVehicle(trip);
Optional<Driver> driverOptional = findOptimalDriver(trip,
vehicleOptional.orElse(null));

// Якщо немає відповідного авто або водія – рейс не створюємо
if (vehicleOptional.isEmpty() || driverOptional.isEmpty()) {
    return false;
}

// Тепер можна забрати заявку з черги
trip = pendingTrips.poll();

Vehicle vehicle = vehicleOptional.get();
Driver driver = driverOptional.get();

// Призначаємо авто і водія на рейс
trip.setVehicle(vehicle);
trip.setDriver(driver);
trip.setStartDate(LocalDate.now());
trip.setPayment(trip.calculatePayment());

// Прибираємо їх зі списків вільних
removeVehicle(vehicle);
removeDriver(driver);

// Додаємо рейс у список активних
tripsInProgress.add(trip);

return true;
}

// Пошук найкращого авто під вагу вантажу
private Optional<Vehicle> findOptimalVehicle(Trip trip) {
    int neededWeight = trip.getNeededWeight();

    return vehicles.stream()
        .filter(v -> v.getStatus() == VehicleStatus.OK)
        .filter(v -> v.getMaxWeight() >= neededWeight)
        .min(Comparator.comparingInt(v -> v.getMaxWeight() -
neededWeight));
}

// Пошук найкращого водія під умови рейсу
private Optional<Driver> findOptimalDriver(Trip trip, Vehicle vehicle) {
    int neededExperience = trip.getNeededExperience();
    int vehicleComplexity = vehicle == null ? 0 :
vehicle.getDrivingComplexity();
    int extraNeed = neededExperience + vehicleComplexity;

    return drivers.stream()
        .filter(d -> d.getExperience() >= extraNeed)
        .min(Comparator.comparingInt(Driver::getExperience));
}

// Завершення рейсу
public boolean completeTrip(Trip trip) {
    if (tripsInProgress.remove(trip)) {

```



```

        trip.setEndDate(LocalDate.now());

        if (trip.getPayment() == null) {
            trip.setPayment(trip.calculatePayment());
        }

        // Додаємо заробіток водію
        trip.getDriver().addEarnings(trip.getPayment());
        trip.getDriver().incrementCompletedTrips();

        // Повертаємо авто і водія в список доступних
        vehicles.add(trip.getVehicle());
        drivers.add(trip.getDriver());

        // Зберігаємо рейс у завершені
        completedTrips.add(trip);

        // Пишемо в лог-файл
        logTrip(trip);
        return true;
    }
    return false;
}

// Статистика рейсів по водіях
public Map<String, Long> getCompletedTripsByDriver() {
    return completedTrips.stream()
        .collect(Collectors.groupingBy(
            t -> t.getDriver().getSurname() + " " +
t.getDriver().getName(),
            Collectors.counting()
        ));
}

// Статистика рейсів по пунктах призначення
public Map<String, Long> getCompletedTripsByDestination() {
    return completedTrips.stream()
        .collect(Collectors.groupingBy(
            t -> t.getAddress().address(),
            Collectors.counting()
        ));
}

// Топ водіїв за заробітком
public List<Driver> getTopEarnersFromHistory() {
    return completedTrips.stream()
        .map(Trip::getDriver)
        .distinct()

.sorted(Comparator.comparingDouble(Driver::getEarnings).reversed())
        .toList();
}

// Вивід статистики у консоль
public void printStatistics() {
    System.out.println("=== Статистика автобази ===");
    System.out.println("Кількість завершених рейсів: " +
completedTrips.size());

    System.out.println("Рейси за водіями:");
    getCompletedTripsByDriver().forEach((driver, count) ->
        System.out.println(driver + " -> " + count));

    System.out.println("Рейси за пунктами призначення:");
    getCompletedTripsByDestination().forEach((destination, count) ->

```

```

        System.out.println(destination + " -> " + count));

        System.out.println("Найбільший заробіток:");
        getTopEarningsFromHistory().forEach(driver ->
            System.out.println(driver.getSurname() + " " +
driver.getName() + " -> " + driver.getEarnings()));
    }
}

```

StreamLogger.java:

```

import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;

public class StreamLogger<T> {
    private final Path logPath;

    public StreamLogger(Path logPath) {
        this.logPath = logPath;
    }

    // Додає один запис у файл
    public void append(T log) {
        try {
            Files.writeString(
                logPath,
                log.toString() + "\n",
                StandardOpenOption.APPEND,
                StandardOpenOption.CREATE
            );
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Main.java:

```

import com.github.javafaker.Faker;
import driver.Driver;
import trip.Address;
import trip.CargoType;
import trip.Trip;
import vehicle.DefaultVehicle;
import vehicle.Vehicle;
import vehicle.VehicleStatus;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.List;
import java.util.Queue;

public class Main {

    // Імена водіїв українською
    private static final String[] DRIVER_NAMES = {
        "Олег", "Іван", "Андрій", "Сергій", "Дмитро",
        "Максим", "Тарас", "Віталій", "Петро", "Юрій"
    };

    // Прізвища водіїв українською
    private static final String[] DRIVER_SURNAMES = {

```

```

        "Коваленко", "Шевченко", "Бондаренко", "Ткаченко", "Мельник",
        "Кравченко", "Савченко", "Петренко", "Лисенко", "Мороз"
    };

    // Міста для маршрутів
    private static final String[] CITIES = {
        "Київ", "Львів", "Одеса", "Харків", "Дніпро",
        "Полтава", "Черкаси", "Вінниця", "Івано-Франківськ", "Чернігів"
    };

    // Вулиці для маршрутів
    private static final String[] STREETS = {
        "Хрещатик", "Шевченка", "Соборна", "Незалежності",
        "Грушевського",
        "Київська", "Лесі Українки", "Садова", "Шкільна", "Центральна"
    };

    // Простий набір зрозумілих автомобілів
    private static final String[] VEHICLES = {
        "Mercedes Sprinter",
        "Mercedes Actros",
        "MAN TGL",
        "MAN TGX",
        "Renault Master",
        "Renault Premium",
        "Volkswagen Crafter",
        "Iveco Daily",
        "DAF XF",
        "Volvo FH"
    };

    public static void main(String[] args) {
        Faker faker = new Faker();

        // Список вільних автомобілів
        List<Vehicle> vehicles = new ArrayList<>();

        // Список вільних водіїв
        List<Driver> drivers = new ArrayList<>();

        // Черга нових заявок на рейси
        Queue<Trip> trips = new ArrayDeque<>();

        // Генеруємо випадкові дані для демонстрації роботи системи
        for (int i = 0; i < 10; i++) {
            // Створюємо автомобіль з назвою, вантажопідйомністю та
            складністю керування
            vehicles.add(new DefaultVehicle(
                VEHICLES[i],
                faker.number().numberBetween(100, 1000),
                faker.number().numberBetween(1, 5),
                VehicleStatus.OK
            ));

            // Створюємо водія
            drivers.add(new Driver(
                DRIVER_NAMES[i],
                DRIVER_SURNAMES[i],
                "+380" + faker.number().digits(9),
                faker.number().numberBetween(1, 20)
            ));

            // Створюємо заявку на перевезення
            trips.add(new Trip(
                faker.number().randomDouble(2, 100, 1000),

```

```

        new Address(
            CITIES[faker.number().numberBetween(0,
CITIES.length - 1)] +
            ", вул. " +
STREETS[faker.number().numberBetween(0, STREETS.length - 1)] +
            ", буд. " +
            faker.number().numberBetween(1, 100)
        ),
        faker.number().numberBetween(50, 500),
        CargoType.values()[faker.number().numberBetween(0,
CargoType.values().length - 1)]
    ));
}

// Створюємо диспетчера з готовими списками
var dispatcher = DispatcherService.builder()
    .drivers(drivers)
    .vehicles(vehicles)
    .pendingTrips(trips)
    .build();

// Запускаємо обробку рейсів
doSomething(dispatcher);
}

public static void doSomething(DispatcherService dispatcherService) {
    // Поки можна призначати нові рейси — призначаємо їх
    while (dispatcherService.assignNextTrip()) {
    }

    // Показуємо всі активні рейси
    System.out.println("\n=== Активні рейси ===");
    dispatcherService.getTripsInProgress().forEach(System.out::println);

    // Показуємо статистику до завершення рейсів
    dispatcherService.printStatistics();

    // Завершуємо всі активні рейси
    var tripsInProgress = dispatcherService.getTripsInProgress();
    for (Trip trip : new ArrayList<>(tripsInProgress)) {
        dispatcherService.completeTrip(trip);
    }

    // Показуємо статистику після завершення рейсів
    dispatcherService.printStatistics();
}
}

```

DriverTest.java:

```

package driver;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class DriverTest {

    // Перевіряємо, що при додаванні грошей збільшується заробіток водія
    @Test
    void addEarnings_shouldIncreaseEarnings() {
        Driver driver = new Driver("Ігор", "Полинько", "0987654321", 5);

        driver.addEarnings(150.5);
    }
}

```

```

        assertEquals(150.5, driver.getEarnings());
    }

    // Перевіряємо, що після завершення рейсу збільшується рахунок виконаних рейсів
    @Test
    void incrementCompletedTrips_shouldIncreaseCounter() {
        Driver driver = new Driver("Ігор", "Полинько", "0987654321", 5);

        driver.incrementCompletedTrips();

        assertEquals(1, driver.getCompletedTrips());
    }
}

```

TripTest.java:

```

package trip;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class TripTest {

    // Перевіряємо, що розрахунок оплати за рейс повертає позитивне значення
    @Test
    void calculatePayment_shouldReturnPositiveValue() {
        Trip trip = new Trip(300.0, new Address("Львів"), 200,
CargoType.FRAGILE);

        double payment = trip.calculatePayment();

        assertTrue(payment > 0);
    }

    // Перевіряємо, що необхідний стаж залежить від параметрів рейсу
    @Test
    void getNeededExperience_shouldDependOnTripData() {
        Trip trip = new Trip(500.0, new Address("Одеса"), 300,
CargoType.DANGEROUS);

        int experience = trip.getNeededExperience();

        assertTrue(experience > 0);
    }
}

```

DefaultVehicleTest.java:

```

package vehicle;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class DefaultVehicleTest {

    // Перевіряємо, що зміна статусу автомобіля працює коректно
    @Test
    void setStatus_shouldChangeStatus() {
        DefaultVehicle vehicle = new DefaultVehicle("Трузовик", 500, 2,

```

```

VehicleStatus.OK);

    vehicle.setStatus(VehicleStatus.BROKEN);

    assertEquals(VehicleStatus.BROKEN, vehicle.getStatus());
}

// Перевіряємо, що геттери повертають коректні значення автомобіля
@Test
void getters_shouldReturnValues() {
    DefaultVehicle vehicle = new DefaultVehicle("Грузовик", 500, 2,
VehicleStatus.OK);

    assertEquals("Грузовик", vehicle.getName());
    assertEquals(500, vehicle.getMaxWeight());
    assertEquals(2, vehicle.getDrivingComplexity());
    assertEquals(VehicleStatus.OK, vehicle.getStatus());
}
}

```

DispatcherServiceTest.java:

```

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import driver.Driver;
import trip.Address;
import trip.CargoType;
import trip.Trip;
import vehicle.DefaultVehicle;
import vehicle.Vehicle;
import vehicle.VehicleStatus;

import static org.junit.jupiter.api.Assertions.*;

class DispatcherServiceTest {

    private DispatcherService dispatcher;

    private Vehicle vehicle;
    private Driver driver;
    private Trip trip;

    @BeforeEach
    void setUp() {
        // Створюємо новий диспетчер перед кожним тестом
        dispatcher = new DispatcherService();

        // Готуємо тестові автомобіль, водія і заявку
        vehicle = new DefaultVehicle("Автомобіль1", 1000, 3,
VehicleStatus.OK);
        driver = new Driver("Ігор", "Полинько", "0987654321", 10);
        trip = new Trip(500.0, new Address("Київ"), 500, CargoType.GENERAL);

        // Додаємо їх в систему
        dispatcher.addVehicle(vehicle);
        dispatcher.addDriver(driver);
        dispatcher.addTrip(trip);
    }

    // Перевіряємо, що рейс успішно назначається, якщо усі ресурси є
    @Test
    void assignNextTrip_success() {
        boolean result = dispatcher.assignNextTrip();
    }
}

```

```

        assertTrue(result);
        assertEquals(1, dispatcher.getTripsInProgress().size());
        assertTrue(dispatcher.getPendingTrips().isEmpty());

        Trip assignedTrip =
dispatcher.getTripsInProgress().iterator().next();
        assertNotNull(assignedTrip.getDriver());
        assertNotNull(assignedTrip.getVehicle());
        assertNotNull(assignedTrip.getStartDate());
        assertNotNull(assignedTrip.getPayment());
    }

    // Перевіряємо, що якщо нема заявок, рейс не призначається
    @Test
    void assignNextTrip_noTrips() {
        dispatcher = new DispatcherService();
        dispatcher.addVehicle(vehicle);
        dispatcher.addDriver(driver);

        boolean result = dispatcher.assignNextTrip();

        assertFalse(result);
    }

    // Перевіряємо, що якщо нема потрібного автомобіля, рейс не призначається
    @Test
    void assignNextTrip_noVehicleForWeight() {
        dispatcher = new DispatcherService();

        Vehicle smallVehicle = new DefaultVehicle("Маленький автомобіль",
100, 1, VehicleStatus.OK);

        dispatcher.addVehicle(smallVehicle);
        dispatcher.addDriver(driver);
        dispatcher.addTrip(trip);

        boolean result = dispatcher.assignNextTrip();

        assertFalse(result);
    }

    // Перевіряємо, що якщо водію не вистачає досвіду, рейс не призначається
    @Test
    void assignNextTrip_noDriverForExperience() {
        dispatcher = new DispatcherService();

        Driver inexperiencedDriver = new Driver("Ігор", "Полинько",
"0987654321", 1);

        dispatcher.addVehicle(vehicle);
        dispatcher.addDriver(inexperiencedDriver);
        dispatcher.addTrip(trip);

        boolean result = dispatcher.assignNextTrip();

        assertFalse(result);
    }

    // Перевіряємо, що після завершення рейсу усі ресурси повертаються назад
    @Test
    void completeTrip_success() {
        dispatcher.assignNextTrip();

        Trip activeTrip = dispatcher.getTripsInProgress().iterator().next();

```

```

        boolean result = dispatcher.completeTrip(activeTrip);

        assertTrue(result);
        assertEquals(1, dispatcher.getCompletedTrips().size());
        assertTrue(dispatcher.getTripsInProgress().isEmpty());
        assertEquals(1, dispatcher.getDrivers().size());
        assertEquals(1, dispatcher.getVehicles().size());
    }

    // Перевіряємо, що завершити рейс, якого нема серед активних, не можна
    @Test
    void completeTrip_notFound() {
        boolean result = dispatcher.completeTrip(trip);

        assertFalse(result);
    }

    // Перевіряємо, що заявка на ремонт змінює статус автомобіля на REPAIR
    @Test
    void driverRequestRepair_changesStatus() {
        dispatcher.assignNextTrip();
        Driver activeDriver =
dispatcher.getTripsInProgress().iterator().next().getDriver();

        dispatcher.driverRequestRepair(activeDriver);

        assertEquals(VehicleStatus.REPAIR,
dispatcher.getTripsInProgress().iterator().next().getVehicle().getStatus());
    }

    // Перевіряємо, що повідомлення о поломці переводить авто у статус BROKEN
    @Test
    void driverReportBrokenVehicle_changesStatus() {
        dispatcher.assignNextTrip();
        Driver activeDriver =
dispatcher.getTripsInProgress().iterator().next().getDriver();

        dispatcher.driverReportBrokenVehicle(activeDriver);

        assertEquals(VehicleStatus.BROKEN,
dispatcher.getTripsInProgress().iterator().next().getVehicle().getStatus());
    }
}

```



```
=== Активні рейси ===
Рейс:
  Автомобіль:
    Назва: Volkswagen Crafter
    Вантажопідйомність: 303
    Складність керування: 2
    Статус: справний
  Водій:
    Ім'я: Олег
    Прізвище: Коваленко
    Телефон: +380716048527
    Стаж: 16
    Заробіток: 0,00 грн
    Виконано рейсів: 0
  Дата початку: 2026-04-23
  Дата завершення: null
  Відстань: 890,94 км
  Пункт призначення: Одеса, вул. Шевченка, буд. 41
  Вага вантажу: 170 кг
  Тип вантажу: крихкий
  Оплата: 90875,88 грн
Рейс:
  Автомобіль:
    Назва: DAF XF
    Вантажопідйомність: 302
    Складність керування: 2
    Статус: справний
  Водій:
    Ім'я: Тарас
    Прізвище: Савченко
    Телефон: +380547977474
    Стаж: 10
    Заробіток: 0,00 грн
    Виконано рейсів: 0
  Дата початку: 2026-04-23
  Дата завершення: null
  Відстань: 407,14 км
```

Рисунок 7.8 – Результат роботи скрипту завдання 2

```
=== Статистика автобази ===
Кількість завершених рейсів: 0
Рейси за водіями:
Рейси за пунктами призначення:
Найбільший заробіток:
=== Статистика автобази ===
Кількість завершених рейсів: 9
Рейси за водіями:
Мельник Дмитро -> 1
Савченко Тарас -> 1
Кравченко Максим -> 1
Ткаченко Сергій -> 1
Коваленко Олег -> 1
Лисенко Петро -> 1
Бондаренко Андрій -> 1
Петренко Віталій -> 1
Мороз Юрій -> 1
Рейси за пунктами призначення:
Вінниця, вул. Шкільна, буд. 43 -> 1
Полтава, вул. Шкільна, буд. 13 -> 1
Київ, вул. Незалежності, буд. 70 -> 1
Івано-Франківськ, вул. Хрещатик, буд. 2 -> 1
Полтава, вул. Лесі Українки, буд. 63 -> 1
Одеса, вул. Хрещатик, буд. 39 -> 1
Дніпро, вул. Лесі Українки, буд. 39 -> 1
Одеса, вул. Хрещатик, буд. 19 -> 1
Одеса, вул. Шевченка, буд. 41 -> 1
Найбільший заробіток:
Мороз Юрій -> 273055.53599999996
Петренко Віталій -> 212862.76200000002
Бондаренко Андрій -> 209776.5
Лисенко Петро -> 178849.044
Мельник Дмитро -> 171042.66
Коваленко Олег -> 90875.88
Кравченко Максим -> 57972.996
```

Рисунок 7.9 – Продовження результату роботи скрипту завдання 2

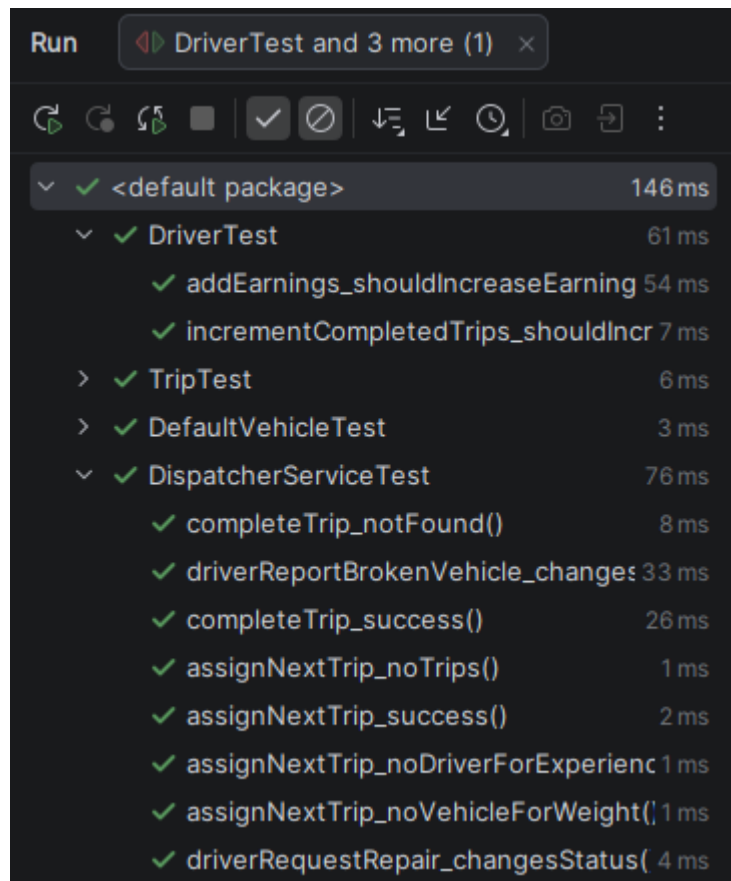


Рисунок 7.10 – Результату роботи тестів завдання 2

Висновок: на цій лабораторній роботі ми отримали навички використання збирача Maven для відвантаження фреймворків, що дозволять скоротити написання програмного коду та тестувати основні модулі створених проектів шляхом покрити їх тестами фреймворку JUnit.